KNOWLEDGE ENGINEERING AND ONTOLOGIES
COURSE PROJECT REPORT

E-Learning System Ontology Development

Department of Computer Engineering
Manisa Celal Bayar University

PREPARED BY

Jalil Guliyev 220315102
Mahammadali Aliyev220315090
Heydar Mehdizade 220315091

1. Executive Summary

This project presents the design, development, and implementation of an E-Learning System Ontology utilizing Semantic Web technologies. The primary objective is to construct a robust knowledge graph that structurally represents academic institutions, users, learning materials, and evaluation processes. The architecture explicitly separates the schema (TBox) and data (ABox) layers, ensuring a scalable and maintainable system. We utilized synthetic data generation techniques via a custom Java application to populate the knowledge graph. Furthermore, we implemented SHACL for strict data validation, SPARQL for advanced querying, and proposed an innovative backend architecture integrating Large Language Models (LLMs) to serve as a natural language interface while mitigating hallucination risks.

2. Project Description

Problem Definition and Motivation: Modern e-learning platforms and university information systems often suffer from fragmented data silos. Information regarding faculties, students, course prerequisites, and grading schemas are usually stored in relational databases that lack semantic interoperability. This makes it difficult to execute complex logical queries, such as inferring a student's academic standing based on deep prerequisite chains.

Objectives and Competency Questions: Our project aims to solve this by modeling the e-learning domain as a formal ontology. The knowledge graph enables machine-readable data integration and logical reasoning. To guide our ontology design and validate its capabilities, we defined the following Competency Questions (CQs) that the system must answer:

- **CQ1:** What is the complete list of registered students and their contact emails?
- **CQ2:** Which specific courses is a given student currently enrolled in?
- **CQ3:** Which courses have a heavy workload (more than 4 credits)?
- **CQ4:** What is the total number of students enrolled in each course?
- **CQ5:** Which advanced courses require a specific fundamental course (e.g., CS-101) as a prerequisite?
- **CQ6:** What is the departmental affiliation of the instructor teaching a specific course?
- **CQ7:** Which students have submitted their assignments after the official deadline?
- **CQ8:** What is the average academic score across all submissions for a specific course?

3. Ontology Design

Our ontology was developed following a structured engineering approach, expanding upon an initial baseline to a comprehensive Version 2 (V2) schema. We utilized Protégé to model the TBox layer.

Core Classes:

- Institution, Faculty, Department for institutional hierarchy.
- Person, subdivided into Student and Instructor (defined as disjoint classes to prevent logical conflicts).

- AcademicContent, encompassing Course, DegreeProgram, and Module (including VideoLecture, ReadingMaterial).
- Assessment (Assignment, Quiz) and Evaluation (Grade).

Object and Data Properties:

- Relationships were established using explicit domains and ranges. For instance, :enrolledIn links Student to Course, while its inverse :hasStudent links Course to Student.
- Data properties were strictly typed using XSD standard datatypes, such as :credits (xsd:integer) and :score(xsd:float).

4. Data Acquisition

Instead of relying on unstructured external datasets that do not perfectly map to our ontology, we developed a Synthetic Data Generation pipeline using the Java ecosystem.

- **Methodology:** A custom backend script (ELearningDataGenerator.java) was developed utilizing the Java Faker library. This application programmatically generated realistic, structured datasets for students, instructors, and courses.
- **Data Formatting:** The Java script ensures that all generated data matches the strict XSD data types required by our OWL ontology (e.g., formatting submission dates to xsd:dateTime). This automated approach guarantees a large volume of syntactically correct data ready for ABox integration.

5. Knowledge Graph Construction

The construction of the Knowledge Graph (KG) followed a strict architectural separation:

- **TBox (Schema):** Saved as elearning-tbox.ttl, containing the class hierarchies and property restrictions.
- **ABox (Data):** Saved as elearning-abox.ttl, containing the instances generated by our Java application. For demonstration and proof-of-concept purposes, a highly interconnected dataset consisting of 14 core individuals (specific students, courses, and grades) was curated.
- **Integration:** The ABox was explicitly imported into the TBox environment within Protégé. A logical reasoner (HermiT/Pellet) was utilized to infer implicit relationships, such as automatically classifying subclasses within the AcademicContent taxonomy.

6. SPARQL Queries

To demonstrate the system's capabilities and answer our Competency Questions, we formulated and executed 10 advanced SPARQL queries.

Q1: Basic Retrieval (All students and emails)

- **Purpose:** To extract a flat list of all registered students and their contact information for administrative overviews.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?name ?email WHERE {
  ?student a :Student ;
           :hasName ?name ;
           :hasEmail ?email .
}
```

Q2: Relational Navigation (Courses a specific student is enrolled in)

- **Purpose:** To demonstrate object property traversal by finding all courses a specific student (e.g., Heydar Mehdizade) is currently taking.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?courseTitle WHERE {
  ?student a :Student ;
           :hasName "Heydar Mehdizade" ;
           :enrolledIn ?course .
  ?course :courseTitle ?courseTitle .
}
```

Q3: Filtering (Courses with more than 4 credits)

- **Purpose:** To show numeric filtering capabilities by retrieving high-workload courses based on integer values.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?courseCode ?title ?credits WHERE {
  ?course a :Course ;
          :courseCode ?courseCode ;
          :courseTitle ?title ;
          :credits ?credits .
  FILTER (?credits > 4)
}
```

Q4: Aggregation (Counting students per course)

- **Purpose:** To utilize aggregate functions (COUNT, GROUP BY) to determine the enrollment capacity and popularity of each course.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?courseTitle (COUNT(?student) AS ?studentCount) WHERE {
  ?course a :Course ;
          :courseTitle ?courseTitle .
  OPTIONAL { ?course :hasStudent ?student }
} GROUP BY ?courseTitle
```

Q5: Hierarchical Query (Advanced courses requiring CS-101)

- **Purpose:** To test the transitive prerequisite chains, ensuring students follow the correct academic path.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?advancedCourseTitle WHERE {
  ?prereq :courseCode "CS-101" .
  ?advancedCourse :hasPrerequisite ?prereq ;
                  :courseTitle ?advancedCourseTitle .
}
```

Q6: Reasoning (Retrieve all Academic Content instances)

- **Purpose:** To verify the reasoner's ability to automatically classify individuals into their superclasses (e.g., recognizing a Course or Module as AcademicContent).

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?content ?type WHERE {
  ?content a :AcademicContent .
  ?content a ?type .
}
```

Q7: Deep Relationship Traversal (Finding the department of a specific course's instructor)

- **Purpose:** To execute a deep graph traversal linking a course to its instructor, and subsequently to the instructor's affiliated academic department.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?instructorName ?dept WHERE {
  ?course :courseTitle "Knowledge Engineering" ;
          :isTaughtBy ?instructor .
  ?instructor :hasName ?instructorName ;
              :belongsToDepartment ?department .
  ?department a ?dept .
}
```

Q8: Date Comparison / Logical Filtering (Late Submissions)

- **Purpose:** To compare xsd:dateTime values to identify students who submitted assignments after the official due date, demonstrating the ontology's capacity for temporal logic.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
SELECT ?studentName ?courseTitle ?submittedDate ?deadline WHERE {
  ?student a :Student ;
           :hasName ?studentName ;
           :submits ?submission .
```

```
  ?submission :submissionFor ?assignment ;
              :submissionDate ?submittedDate .
  ?course :hasAssignment ?assignment ;
          :courseTitle ?courseTitle .
  ?assignment :dueDate ?deadline .
  FILTER (?submittedDate > ?deadline)
}
```

Q9: Advanced Aggregation (Calculating Average Score per Course)

- **Purpose:** To use the AVG function to calculate the mean academic performance across all student submissions for a specific course.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT ?courseTitle (AVG(?score) AS ?averageScore) WHERE {
  ?course a :Course ;
          :courseTitle ?courseTitle ;
          :hasAssignment ?assignment .
  ?submission :submissionFor ?assignment ;
              :hasGrade ?grade .
  ?grade :score ?score .
} GROUP BY ?courseTitle
```

Q10: Chain Relationship Extraction (All Students Taught by a Specific Instructor)

- **Purpose:** To map the sphere of influence of an instructor by chaining the teaches and enrolledIn relationships without needing complex SQL JOIN operations.

Kod snippet'i

```
PREFIX : <http://www.example.org/elearning-ontology#>
SELECT DISTINCT ?instructorName ?studentName WHERE {
  ?instructor a :Instructor ;
              :hasName "Dr. Alan Smith" ;
              :teaches ?course .
  ?student :enrolledIn ?course ;
           :hasName ?studentName .
}
```

7. Validation (SHACL)

To ensure data consistency and prevent logical errors during data entry, we implemented Shapes Constraint Language (SHACL). Key constraints include:

- **Cardinality Restrictions:** The CourseShape dictates that a course must be taught by exactly one instructor (sh:minCount 1 ; sh:maxCount 1).
- **Property & Class Constraints:** The StudentShape enforces that every student must be enrolled in at least one valid :Course.
- **Data Type Validation:** The GradeShape ensures that scores are strictly floats and limits the value range between 0.0 and 100.0 (sh:minInclusive 0.0 ; sh:maxInclusive 100.0), preventing invalid academic records.

8. LLM Integration and Innovation

To make our semantic web application accessible to non-technical users and automate data population, we designed a conceptual backend microservice architecture that integrates a Large Language Model (LLM).

- **Workflow:** A user submits a natural language query via the frontend. The backend intercepts this request, forwards it to the LLM for Natural Language-to-SPARQL translation, executes the query against our RDF Knowledge Graph, and returns the deterministic data.
- **Hallucination Mitigation Strategy:** LLMs are prone to generating non-existent properties. To prevent this, we employ a **Schema-Prompting & RAG** approach. During prompt construction, we inject a simplified version of our TBox schema into the LLM context. The LLM is strictly restricted to query compilation. Data retrieval is exclusively handled by the secure backend executing against the ABox. If the LLM generates syntactically incorrect SPARQL, the backend exception handling catches the error and provides a fallback response, ensuring no fabricated academic data is ever returned.

9. Evaluation, Discussion, and Conclusion

Evaluation & Strengths: The developed E-Learning Ontology successfully meets all project requirements. The strict separation of TBox and ABox allows for high modularity. The integration of SHACL provides robust data validation, acting as an effective semantic firewall. Developing a custom Java data generator ensures that the system can be scaled to handle thousands of records effortlessly.

Limitations: While the architecture is highly scalable, the current ABox demonstration utilizes a constrained dataset (14 individuals). This was a deliberate choice to ensure clarity during the proof-of-concept presentation, but a larger dataset is required for comprehensive stress-testing of the reasoner's performance.

Conclusion & Future Work (Production Vision): This project establishes a solid semantic foundation for a modern university backend system. In future iterations, we plan to transform this architecture into a fully functional, role-based web application. By integrating Apache Jena and building a robust REST API layer with Spring Boot, the system will support two distinct user authorizations:

- **Admin Role:** Authorized personnel (e.g., faculty administrators) will have the ability to dynamically add new instances (such as enrolling students, creating new courses, and assigning grades) directly into the knowledge graph via a secure web interface.
- **User (Student) Role:** Students will interact with a Natural Language Interface to query their personal academic standing, schedules, and grades without needing to understand SPARQL.

This role-based separation of data entry and querying will fully realize the potential of our semantic web infrastructure as a scalable, real-world educational platform.

10. References

1. Antoniou, G., & van Harmelen, F. (2004). *A Semantic Web Primer*. MIT Press.

2. W3C. (2012). *OWL 2 Web Ontology Language Document Overview*. Retrieved from https://www.w3.org/TR/owl2-overview/

3. W3C. (2013). *SPARQL 1.1 Query Language*. Retrieved from https://www.w3.org/TR/sparql11-query/

4. W3C. (2017). *Shapes Constraint Language (SHACL)*. Retrieved from https://www.w3.org/TR/shacl/

5. Java Faker. (n.d.). *DiUS/java-faker: Brings blah blah to your java projects*. GitHub. Retrieved from https://github.com/DiUS/java-faker
[cite: 1]